Short Topic Summary: for + if/else

Oğuzhan Işılay

2026-01-12

Short Topic Summary: for + if/else

1. For Loop

In R, a **for loop** allows us to repeat the same operation for multiple values.

Example structure:

```
items <- c(1,2,3)

for (i in items) {
  # operation to be performed on each item
}
```

items → a vector or list containing multiple values
i → represents one value from the list during each iteration

The loop follows this logic:

Take each element inside **items**, one by one, and execute the code inside the loop.

Example vector:

```
leaf_lengths <- c(3.2, 5.1, 4.0)
```

The function **c()** means *combine*.
It combines multiple values into a vector.

Here we created a vector named **leaf_lengths**.

Now we can print each value using a loop:

```
for (u in leaf_lengths) {
  print(u)
}
```

```
## [1] 3.2
## [1] 5.1
## [1] 4
```

The computer will:

First iteration → print first value
Second iteration → print second value
Third iteration → print third value

The loop stops when the vector ends.

Creating an Empty Vector

Sometimes we need a place to store results.

```
results <- c()
```

This creates an **empty vector**.

Daily life analogy:

Imagine an empty basket where we will place our results.

Now we process the data:

```
for (u in leaf_lengths) {
  results <- c(results, u * 2)
  print(results)
}
```

```
## [1] 6.4
## [1]  6.4 10.2
## [1]  6.4 10.2  8.0
```

What happens here?

Each value of **leaf_lengths** is taken one by one.
Each value is multiplied by 2.
The result is added to the **results** vector.

So the loop is:

Take value → process it → store the result.

This structure is very common in biological data analysis.

---

2. if / else

The **if / else** structure allows the computer to make decisions.

Basic logic:

If something is true → do one thing
Otherwise → do another thing

General structure:

'if (condition) { # code if true }'

The condition must return **TRUE** or **FALSE**.

Example:

```
leaf_length <- 6

if (leaf_length > 5) {
  print("Long leaf")
}
```

```
## [1] "Long leaf"
```

Since 6 > 5 is TRUE, the message is printed.

If the value was 4, the condition would be FALSE and nothing would happen.

---

if / else structure

'if (condition) { # if true } else { # if false }'

Example:

```
leaf_length <- 4

if (leaf_length > 5) {
  print("Long leaf")
} else {
  print("Short leaf")
}
```

```
## [1] "Short leaf"
```

Since 4 > 5 is FALSE, the **else** block runs.

Output:

Short leaf

Multiple Conditions

Sometimes we need more than two conditions.

```
if (leaf_length < 3) {
  print("Very short")
} else if (leaf_length < 6) {
  print("Medium")
} else {
  print("Long")
}
```

```
## [1] "Medium"
```

The computer checks conditions step by step.

First condition
If FALSE → move to next condition
Continue until one condition becomes TRUE.

---

3. Using for with if / else

Often we combine **loops** and **conditions**.

for → processes values one by one
if / else → decides what to do with each value

Example:

```r
leaves <- c(2.5, 4.8, 6.1)

for (u in leaves) {
  if (u > 5) {
    print("Long leaf")
  } else {
    print("Short leaf")
  }
}
```

```
## [1] "Short leaf"
## [1] "Short leaf"
## [1] "Long leaf"
```

The logic is:

Take a leaf
Check its length
Print the classification
Move to the next leaf

This represents the core logic behind many biological data analyses.

Mini Example: GC% and Motif Analysis in DNA Sequences

Example 1: Checking GC Content in a DNA Sequence

We examine whether each nucleotide forms a **GC strong bond** or an **AT weak bond**.

```r
DNA_sequence <- c("A", "G", "C", "T", "G")

for (nucleotide in DNA_sequence) {

  if (nucleotide == "G" | nucleotide == "C") {
    print(paste(nucleotide, ": GC Strong Bond"))
  } else {
    print(paste(nucleotide, ": AT Weak Bond"))
  }

}
```

```
## [1] "A : AT Weak Bond"
## [1] "G : GC Strong Bond"
## [1] "C : GC Strong Bond"
## [1] "T : AT Weak Bond"
## [1] "G : GC Strong Bond"
```

Example 2: Simple Motif Search

We have several DNA regions and want to detect the **start codon (ATG)**.

```r
regions <- c("CCG", "ATG", "TTA", "ATG")

for (i in 1:length(regions)) {

  if (regions[i] == "ATG") {
    print(paste(i, ". region contains a start codon!"))
  } else {
    print(paste(i, ". region is clean."))
  }

}
```

```
## [1] "1 . region is clean."
## [1] "2 . region contains a start codon!"
## [1] "3 . region is clean."
## [1] "4 . region contains a start codon!"
```

Example 3: GC Ratio Calculation

We count how many **G** and **C** bases exist in the sequence.

```r
dna <- c("G", "C", "A", "T", "G", "G", "A")

gc_number <- 0
```

```r
for (n in dna) {
  if (n == "G" | n == "C") {
    gc_number <- gc_number + 1
  }
}

total_length <- length(dna)

gc_ratio <- (gc_number / total_length) * 100

print(paste("Total GC Count:", gc_number))
```

```
## [1] "Total GC Count: 4"
```

```r
print(paste("GC Ratio:", gc_ratio, "%"))
```

```
## [1] "GC Ratio: 57.1428571428571 %"
```

Example 4: Multiple Conditions (Temperature and Enzyme Activity)

We evaluate enzyme activity at different temperatures.

```r
temperatures <- c(10, 37, 55, 90)

for (t in temperatures) {

  if (t < 20) {
    print(paste(t, "degrees: Enzyme is very slow."))
  } else if (t >= 20 & t <= 45) {
    print(paste(t, "degrees: Optimum working temperature."))
  } else {
    print(paste(t, "degrees: Enzyme structure may be damaged (Denaturation)."))
  }

}
```

```
## [1] "10 degrees: Enzyme is very slow."
## [1] "37 degrees: Optimum working temperature."
## [1] "55 degrees: Enzyme structure may be damaged (Denaturation)."
## [1] "90 degrees: Enzyme structure may be damaged (Denaturation)."
```